

Claude Code

من الصفر إلى الاحتراف

دليل عملي لبناء المشاريع وفهم الكود
وإدارة التطوير باستخدام Claude Code

للمطور العربي



تعلّم
جرب
ابن
تطور



برومبتات عربية
t.me/PromptsArabic

برومبتات عربية

جميع الحقوق محفوظة لقناة برومبتات عربية.

هذا المنتج مخصص للاستخدام الشخصي للمشتري. لا يجوز إعادة نشره أو بيعه أو توزيعه أو اقتباس أجزاء منه لأغراض تجارية دون إذن كتابي من المالك.

القناة الرسمية: <https://t.me/PromptsArabic>

اسم المنتج	Claude Code من الصفر إلى الاحتراف
الإصدار	الإصدار الأول
اللغة	العربية
المقاس	A4 عمودي — مهيأ للقراءة على الهاتف
تمت مراجعة معلومات وميزات 6 سبتمبر 2026 هذا الدليل حتى تاريخ	
مصادر التحقق	code.claude.com/docs (overview, setup, permission-modes, memory, sessions, commands, security)

ملاحظة مهمة عن تاريخ المحتوى

منصات الذكاء الاصطناعي تتغير بسرعة. كل ميزة مذكورة هنا تم التحقق منها من المصدر الرسمي بتاريخ 6 سبتمبر 2026. إذا مر وقت طويل على هذا التاريخ، راجع الصفحة الرسمية للميزة قبل الاعتماد عليها في عمل حساس.

عن هذا الدليل

هذا الدليل ليس شرحاً لأوامر سطر أوامر Claude Code أداة برمجة وكيلة تقرأ مشروعك وتعديل ملفاته وتشغل الأوامر، ولهذا فإن أهم ما ستتعلمه هنا ليس كيف تطلب منها، بل كيف تتحكم فيها: ما الذي تسمح به، وما الذي تراجععه، وكيف تتراجع عن تغيير، وكيف تبني ذاكرة للمشروع لا تعيد شرحه كل يوم. كل أمر في هذا الدليل مأخوذ من التوثيق الرسمي وقت المراجعة.

لمن هذا الدليل

- المبرمجون الذين جربوا Claude Code وشعروا أنهم لا يسيطرون عليه
- طلاب مشاريع التخرج الذين يعملون على مستودع واحد لأشهر
- من يريد استخدامه في مشروع فيه كود قائم لا في مشروع فارغ
- من يهيمه الأمان: الأسرار، ومراجعة التغييرات، والتراجع
- من يريد الانتقال من الاستخدام العشوائي إلى سير عمل منظم

ماذا ستتعلم

- التثبيت الرسمي الصحيح على Windows و macOS و Linux، والتحديث والتشخيص
- أوضاع الأذونات الستة ومتى يصلح كل واحد، وما لا يوافق عليه أي وضع
- نظام الذاكرة: ملفات CLAUDE.md وقواعد المسارات والذاكرة التلقائية
- إدارة الجلسات: الاستئناف والتفريع والضغط والتصدير
- التراجع عن تغييرات أحدثها الوكيل، ومراجعة الفروق قبل الاعتماد
- Git والاختبارات والتصحيح وإعادة الهيكلة بطريقة قابلة للمراجعة
- المهارات والخطافات والوكلاء الفرعيين و MCP والإضافات
- سير عمل كامل على عشر مراحل من البيئة إلى الإصدار

قبل أن تبدأ

المتطلبات

- جهاز Windows أو macOS أو Linux مع صلاحية تثبيت برامج
- معرفة أساسية بـ Git والطرفية Terminal
- اشتراك Claude مدفوع أو حساب Anthropic Console — معظم الواجهات تتطلب ذلك
- مشروع برمجي حقيقي ولو صغيرا، لا مجلد فارغ
- على Windows الأصلي: ينصح بتثبيت Git for Windows

كيف تستخدم هذا الدليل

- نفذ كل أمر بنفسك على مشروع حقيقي أثناء القراءة
- لا تتجاوز فصل الأذونات — هو الفصل الذي يحميك من أضرار حقيقية
- الأوامر داخل الصناديق السوداء تكتب في الطرفية كما هي
- ما بين قوسين مربعين مثل [اسم المشروع] تستبدله بقيمتك
- ارجع إلى الفصل الأخير كورقة مرجعية بعد أن تتعلم الأساس

طريقة القراءة الموصى بها

لا تقرأ الدليل كله دفعة واحدة. اقرأ فصلا واحدا، ثم نفذ تمرينه على مهمة حقيقية من عملك، ثم انتقل. أربعة فصول مطبقة أفضل من أربعة عشر فصلا مقروءا.

10	● ما هو Claude Code ومتى تستخدمه
10	الفرق بينه وبين تطبيق Claude
10	الواجهات المتاحة
11	ما الذي يجيده فعلا
11	متى لا تستخدمه
12	● التثبيت والتحقق والتحديث
12	macOS و Linux و WSL
12	Windows
13	طرق تثبيت أخرى رسمية
13	التحديث
14	أول تشغيل والتحقق
15	● الجلسة الأولى: من فتح المشروع إلى أول تعديل
15	الخطوات الخمس الأولى
16	أول ثلاثة طلبات في أي مشروع جديد
17	الأوامر التي ستستخدمها يوميا
18	● أوضاع الأذونات: أهم فصل في الدليل
18	الأوضاع الستة
18	كيف تبدل الوضع
19	في أي وضع تبدأ جلستك
19	ما لا يوافق عليه أي وضع تلقائيا

20	الوضع الخطر ومتى يجوز
20	بديل عملي أفضل من تخطي الأذونات
21	● الذاكرة: CLAUDE.md وقواعد المسارات والذاكرة التلقائية
21	الفرق بين الآليتين
22	أين توضع ملفات CLAUDE.md
22	كيف تبدأ
22	كيف تكتب تعليمات فعالة
23	متى تضيف إلى CLAUDE.md
23	استيراد ملفات أخرى
24	قواعد المسارات في <code>claude/rules</code> .
24	الذاكرة التلقائية
25	إذا لم يلتزم Claude بتعليماتك
26	● الجلسات: الاستئناف والتفريع وإدارة السياق
26	الاستئناف
26	ما الذي يستعاد فعلاً
27	تسمية الجلسات
27	تفريع محادثة
27	إدارة السياق داخل الجلسة
28	أين تحفظ الجلسات
29	● مراجعة التغييرات والتراجع عن خطأ
29	قاعدة المراجعة قبل الاعتماد
29	نقاط الاستعادة Checkpointing
29	شبكة أمان لا تعتمد على الأداة

30	حدود مجلد العمل
31	● العمل مع Git والفروع وطلبات الدمج
31	سير عمل الفرع الواحد
31	طلبات مفيدة مع Git
32	الأتمة في التكامل المستمر
33	● الاختبارات والفحص والتصحيح وإعادة الهيكلة
33	الاختبارات أولاً
34	التصحيح
35	إعادة الهيكلة
35	الفحص والتنسيق
35	المراجعة الأمنية
36	● المهارات Skills والخطافات Hooks
36	المهارات
36	الخطافات
37	الإضافات Plugins
38	● الوكلاء الفرعيون والعمل المتوازي وأشجار العمل
38	الوكلاء الفرعيون Subagents
38	أشجار العمل Worktrees
39	مقي تستخدم كل آلية
40	● MCP: ربط Claude Code بأدواتك
40	من أين تبدأ
40	أمان MCP

40	قاعدة عملية
42	● الأمان: الأسرار ومراجعة الأوامر والمحتوى غير الموثوق
42	الحمايات المدمجة
42	حقن التعليمات Prompt Injection
43	الأسرار: القاعدة التي لا تكسر
44	العمل على كود حساس
45	● إدارة السياق والمشاريع الكبيرة
45	ماذا يملأ سياقك
45	خمس عادات تقلل استهلاك السياق
45	المستودعات الكبيرة
46	تقسيم المهمة الكبيرة
47	● سير عمل كامل: من البيئة إلى الإصدار
47	المرحلة صفر: البيئة
47	المرحلة الأولى: المتطلبات
48	المرحلة الثانية: المعمارية
48	المرحلة الثالثة: الأساس
48	المرحلة الرابعة: الميزات الأساسية
48	المرحلة الخامسة: الاختبارات
49	المرحلة السادسة: تجربة الاستخدام
49	المرحلة السابعة: الأمان
49	المرحلة الثامنة: البناء
49	المرحلة التاسعة: مراجعة الإصدار
51	● ورقة المراجعة السريعة

51	التثبيت والتحديث
51	الجلسات
52	الأوامر داخل الجلسة
54	أوضاع الأذونات
54	روتين الجلسة الآمنة
54	أخطاء تجنبها

ما هو Claude Code ومتى تستخدمه

الوصف الرسمي في التوثيق: أداة برمجة وكيلة تقرأ الكود وتعديل الملفات وتشغل الأوامر وتتكامل مع أدوات التطوير، متاحة في الطرفية والمحرر وتطبيق الحاسوب والمتصفح. الكلمة المفتاحية هنا «وكيلة»: هي لا تقترح عليك كودا لتنسخه، بل تنفذ في مشروعك.

الفرق بينه وبين تطبيق Claude

الأداة	الحاجة
تطبيق Claude	شرح مقطع كود نسخته
Claude Code	فهم مشروع كامل وتعديل ملفاته
تطبيق Claude	كتابة محتوى أو تحليل مستند
Claude Code	إصلاح خطأ يمتد عبر عدة ملفات
Claude Code	إنشاء فرع وcommit وطلب دمج

الواجهات المتاحة

التوثيق يذكر أن Claude Code يعمل على عدة واجهات، وكلها تتصل بالمحرك نفسه، فملفات CLAUDE.md والإعدادات وخوادم MCP تعمل عبرها جميعا:

- **الطرفية Terminal**: الواجهة الكاملة، وهي التي يركز عليها هذا الدليل.
- **VS Code**: إضافة تعرض الفروق داخل المحرر ومراجعة الخطة وسجل المحادثات.
- **JetBrains**: إضافة لـ IntelliJ و PyCharm و WebStorm وغيرها، وتتطلب تثبيت الـ CLI منفصلا.
- **تطبيق الحاسوب**: تطبيق مستقل لمراجعة الفروق بصريا وتشغيل عدة جلسات جنبا إلى جنب.
- **المتصفح**: تشغيل المهام الطويلة بلا إعداد محلي على claude.ai/code.

ما الذي يتطلب اشتراكا

التوثيق يذكر أن معظم الواجهات تتطلب اشتراك Claude أو حساب Anthropic Console، وأن تطبيق الحاسوب يتطلب اشتراكا مدفوعا. كما تدعم واجهات الطرفية و VS Code و JetBrains مزودين خارجيين.

ما الذي يجيده فعلا

- أتمتة العمل المؤجل: كتابة اختبارات لكود بلا اختبارات، وإصلاح أخطاء الفحص، وحل تعارضات الدمج، وتحديث الاعتماديات.
- بناء ميزات وإصلاح أخطاء: يخطط، ثم يكتب عبر عدة ملفات، ثم يتحقق.
- العمل مع Git مباشرة: تجهيز التغييرات وكتابة رسائل ال commit وإنشاء الفروع وفتح طلبات الدمج.
- الربط بأدواتك عبر MCP، والتخصيص بالمهارات والخطافات.

متى لا تستخدمه

لا تستخدمه على كود لا تفهمه ولا تنوي مراجعته. الأداة تنفذ، والمسؤولية عن مراجعة ما نفذ تبقى عليك. وهذه ليست نصيحة عامة: التوثيق نفسه يضع بندا اسمه مسؤولية المستخدم، ويقول إن Claude Code لا يملك إلا الصلاحيات التي تمنحها، وأنتك مسؤول عن مراجعة الكود والأوامر المقترحة قبل الموافقة.

التثبيت والتحقق والتحديث

التوثيق يوصي بالمتبث الأصلي Native Install. المسارات التالية منقولة من صفحة التوثيق الرسمية وقت مراجعة هذا الدليل.

WSL و Linux و macOS

المتبث الأصلي – الطريقة الموصى بها

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows

PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

موجه الأوامر CMD

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del  
install.cmd
```

أي صدف أنت فيها

التوثيق يعطي علامة مميزة: إذا ظهرت رسالة أن `&&` ليست فاصل جمل صحيحا فأنت في PowerShell لا في CMD. وإذا ظهر أن `irm` أمر غير معروف فأنت في CMD لا في PowerShell. موجه PowerShell يبدأ بـ `PS C:` بينما موجه CMD يبدأ بـ `C:\` بلا `PS`.

Git for Windows على ويندوز الأصلي

التوثيق ينصح بتثبيت Git for Windows على ويندوز الأصلي حتى يستطيع Claude Code استخدام أداة Bash. وبدونه يستخدم PowerShell كأداة الصدف. أما إعدادات WSL فلا تحتاج Git for Windows.

طرق تثبيت أخرى رسمية

الطريقة	الأمر	التحديث التلقائي
Homebrew — القناة المستقرة	<code>brew install --cask claude-code</code>	لا
Homebrew — أحدث إصدار	<code>brew install --cask claude-code@latest</code>	لا
WinGet	<code>winget install Anthropic.ClaudeCode</code>	لا
المثبت الأصلي	أوامر <code>curl</code> و <code>irm</code> أعلاه	نعم، في الخلفية

التوثيق يوضح أن قناة Homebrew المستقرة تتأخر عادة نحو أسبوع وتتخطى الإصدارات التي فيها انحدارات كبيرة، بينما `claude-code@latest` يتابع أحدث قناة. كما يمكن التثبيت عبر مديري حزم لينكس: `apt` و `dnf` و `apk` على Debian و Fedora و RHEL و Alpine.

التحديث

تحديث Homebrew

```
brew upgrade claude-code
brew upgrade claude-code@latest
```

تحديث WinGet

```
winget upgrade Anthropic.ClaudeCode
```

تحديث تثبيت npm

```
npm install -g @anthropic-ai/claude-code@latest
```

خطآن شائعان في تثبيت npm

التوثيق يحذر صراحة من `sudo npm install -g` لأنه يؤدي إلى مشكلات صلاحيات ومخاطر أمنية. ويحذر أيضا من `npm update -g` لأنه يلتزم بنطاق الإصدارات الذي حدد وقت التثبيت الأول وقد لا ينقلك إلى أحدث إصدار. استخدم `@latest` كما في الأمر أعلاه.

أول تشغيل والتحقق

بدء العمل في مشروعك

```
cd path/to/your-project
claude
```

سيطلب منك تسجيل الدخول في أول استخدام. وإذا كان متغير البيئة `ANTHROPIC_API_KEY` مضبوطاً، فإن Claude Code يتخطى شاشة الدخول ويطلب منك الموافقة على المفتاح بدلاً منها.

فحص البيئة والتشخيص

```
claude doctor
```

`claude doctor` ما زال أمراً رسمياً وقت مراجعة هذا الدليل: صفحة الإعدادات تذكره في سياق إدارة المثبت الأصلي للمشغل في `~/.local/bin/claude` كرابط رمزي إلى `~/.local/share/claude/versions/`، وتقول إن `claude doctor` يبلغ عن مشغل لم ينشئه المثبت الأصلي. ويوجد أيضاً أمر `/doctor` داخل الجلسة، ويصفه التوثيق بأنه فحص إعدادات يشخص المشكلات ويستطيع إصلاحها.

الجلسة الأولى: من فتح المشروع إلى أول تعديل

الخطأ الأول الذي يقع فيه معظم المستخدمين: يفتحون الأداة ويطلبون منها بناء ميزة فوراً. الترتيب الصحيح: افهم، ثم خطط، ثم نفذ، ثم راجع.

الخطوات الخمس الأولى

1 افتح المشروع

من داخل مجلد المشروع، شغل `claude`. لا تشغله من مجلد المنزل.

2 افهم قبل أن تعدل

اطلب شرحاً للبنية قبل أي طلب تعديل.

3 ادخل وضع التخطيط

استخدم `Shift+Tab` للتنقل بين الأوضاع، أو `/plan`، لتحصل على خطة قبل التعديل.

4 راجع الخطة

اقرأها فعلاً. تعديل الخطة أرخص بكثير من تعديل الكود.

5 نفذ ثم راجع الفروق

بعد التنفيذ، شغل `/diff` لمراجعة ما تغير في شجرة العمل.

أول ثلاثة طلبات في أي مشروع جديد

الطلب الأول – فهم البنية

اشرح لي بنية هذا المشروع.

ما أريده:

أولاً. ما الذي يفعله هذا المشروع، في ثلاث جمل
ثانياً. نقطة الدخول الرئيسية وكيف يبدأ التنفيذ
ثالثاً. أهم خمسة مجلدات ووظيفة كل واحد
رابعاً. كيف تشغل الاختبارات وكيف يبني المشروع
خامساً. ثلاثة أشياء غير متوقعة أو غير قياسية في هذا المستودع

لا تعدل أي ملف. اقرأ فقط.

الطلب الثاني – تتبع مسار

تتبع ما يحدث عندما ينفذ المستخدم:

[الإجراء]

اذكر بالترتيب: الملفات التي يمر بها التنفيذ، والدوال الرئيسية في كل ملف،
وأين تتخذ القرارات المهمة، وأين قد يفشل المسار.

اذكر مسار كل ملف ورقم السطر عند الإشارة إلى كود.
لا تعدل شيئاً.

الطلب الثالث – أول تعديل صغير

المطلوب:

[التعديل المطلوب]

قبل أن تكتب أي كود:

أولاً. اعرض خطة: ما الملفات التي ستعدلها وماذا في كل ملف
ثانياً. اذكر ما قد ينكسر بسبب هذا التعديل
ثالثاً. انتظر موافقتي

بعد الموافقة نفذ، ثم اشرح ما فعلته في خمسة أسطر.

الأوامر التي ستستخدمها يوميا

الأمر	ماذا يفعل
help/	عرض المساعدة والأوامر المتاحة
init/	إنشاء ملف CLAUDE.md للمشروع
plan/	الدخول في وضع التخطيط من الموجه مباشرة
diff/	مراجعة التغييرات في شجرة العمل
context/	عرض ما يستهلك السياق حاليا كشبكة ملونة
compact/	تفريغ السياق بتلخيص المحادثة
clear/	بدء محادثة جديدة بسياق فارغ
permissions/	إدارة قواعد السماح والسؤال والمنع
model/	تبديل النموذج وحفظه كافتراضي
doctor/	فحص إعدادات الشخص المشكلات ويستطيع إصلاحها

أوضاع الأذونات: أهم فصل في الدليل

وضع الأذونات يحدد ما الذي يستطيع Claude فعله دون أن يسألك. فهم هذا الفصل هو الفرق بين أداة تحت سيطرتك وأداة تفاجئك.

الأوضاع الستة

الوضع	ما يعمل بلا سؤال	الأنسب لـ
Manual (قيمه) (default)	يسأل قبل معظم ما يعدل ملفاً أو يشغل أمراً أو يصل للشبكة	العمل الحساس والكود غير المألوف
acceptEdits	القرارات وتعديلات الملفات وأوامر نظام ملفات شائعة	التكرار على كود تراجعه
plan	القرارات، مع أوامر يوافق عليها المصنف عند توفر وضع auto	استكشاف مستودع قبل تعديله
auto	كل شيء، مع فحوص أمان في الخلفية	المهام الطويلة وتقليل إرهاق الأسئلة
dontAsk	الأدوات المعتمدة مسبقاً فقط	التكامل المستمر والسكربتات المقيدة
bypassPermissions	كل شيء	الحاويات والأجهزة الافتراضية المعزولة فقط

لماذا اسمان لوضع واحد

الوضع الذي يراجع كل إجراء اسمه Manual في الـ CLI وفي `claude --help` وفي إضافتي VS Code و JetBrains وفي تطبيق الحاسوب، لكن قيمته في الإعدادات هي `default`، وهي القيمة التي تستخدمها الخطافات وتكاملات الـ SDK. ويقبل الـ CLI الاسم `manual` كمرادف، مثل `claude --permission-mode manual`.

كيف تبدل الوضع

- داخل الـ CLI: مفتاح `Shift+Tab`.

- في VS Code: مؤشر الوضع. وفي تطبيق الحاسوب: محدد الوضع.
- عند التشغيل: `claude --permission-mode plan` مثلا.
- من ملف إعدادات: المفتاح `permissions.defaultMode`.

في أي وضع تبدأ جلستك

التوثيق يوضح أن Claude Code يأخذ الوضع من أول ما ينطبق: العلم `--permission-mode` أو `--dangerously` `skip-permissions`، ثم `permissions.defaultMode` في ملف إعدادات، ثم الافتراضي المدمج. وعلى خط Pro و Max و Team في الطرفية أو عبر إضافة VS Code، الافتراضي المدمج هو وضع `auto`. أما مع `claude -p` أو الـ Agent SDK أو خطة Enterprise أو مفتاح Console فالافتراضي هو `default`.

قيمتان لا تعملان من ملفات إعدادات المشروع

التوثيق يذكر أن ضبط `"auto"` داخل `.claude/settings.json` أو `.claude/settings.local.json` لا يسري، وأن ضبط `"bypassPermissions"` في هذين الملفين لا يسري أيضا وتبدأ الجلسة في وضع Manual. بقية القيم تسري من أي ملف إعدادات.

ما لا يوافق عليه أي وضع تلقائيا

هذه القائمة من التوثيق مهمة لأنها تعني أن حتى أخطر وضع يبقى فيه سياج. لا يوافق Claude Code تلقائيا في أي وضع، بما في ذلك `bypassPermissions`، على:

- الأدوات التي تطابق قاعدة سؤال صريحة.
- الأدوات التي تتطلب تفاعل المستخدم مثل أداة `AskUserQuestion`.
- عمليات `rm` و `rmdir` التي تستهدف مسارا حرجا — ولا توافق عليها قاعدة سماح ولا خطاف.
- ضمانات المراسلة بين الجلسات.

ويذكر التوثيق أيضا أن قواعد المنع `deny` تمنع في كل وضع بما فيه `bypassPermissions`، بينما قواعد السماح `allow` لا أثر لها داخل `bypassPermissions`.

الوضع الخطر ومتى يجوز

قبل أن تفكر في تخطي الأذونات

الوضع `bypassPermissions` ، ومع العلم `--dangerously-skip-permissions` ، ليس إعدادا افتراضيا ولا اختصارا للراحة. التوثيق يشترط لتشغيل مهمة غير مراقبة بهذا الوضع عزلا حقيقيا: حاوية أو جهاز افتراضي أو بيئة تشغيل معزولة، وعلى لينكس و macOS تشغيله بمستخدم غير الجذر. لا تستخدمه على جهازك الشخصي وعلى مستودع فيه أسرار أو صلاحيات نشر.

بديل عملي أفضل من تخطي الأذونات

إذا كان دافعك كثرة الأسئلة، فالتوثيق يعرض مسارا أنظف: وضع Manual مع صندوق Bash الرملي في وضع السماح التلقائي. تشغيله بـ `claude --permission-mode default` ثم `/sandbox` وتختار `auto-allow`. وتبقى قواعد المنع سارية، وتبقى قواعد السؤال التي تسمي أمرا بعينه — مثل `Bash(git push *)` — تسأل.

أمثلة إطلاق بأوضاع مختلفة

```
claude --permission-mode default
claude --permission-mode plan
claude -p "run the test suite" --permission-mode dontAsk \
  --allowedTools "Bash(npm test)" "Read"
```

الذاكرة: CLAUDE.md وقواعد المسارات والذاكرة التلقائية

كل جلسة تبدأ بنافذة سياق جديدة. آليتان تنقلان المعرفة بين الجلسات: ملفات CLAUDE.md التي تكتبها أنت، والذاكرة التلقائية التي يكتبها Claude بنفسه.

الفرق بين الآليتين

الذاكرة التلقائية	CLAUDE.md
Claude	أنت من يكتبها
دروس وأنماط	تعليمات وقواعد ماذا تحتوي
لكل مستودع، ومشاركة عبر أشجار العمل	مشروع أو مستخدم أو مؤسسة النطاق
كل جلسة، أول 200 سطر أو 25 كيلوبايت	كل جلسة تحمل في
تفضيلاتك وتصحيحاتك وسياق لا يستنتج من الكود	معايير الكود وسير العمل ومعمارية المشروع استخدمها لـ

الذاكرة سياق لا إعداد ملزم

التوثيق صريح: Claude يتعامل مع الملفين كسياق لا إعداد مفروض. لمنع إجراء مهما قرر Claude، استخدم خطاب `PreToolUse` بدلا من كتابة قاعدة في CLAUDE.md.

أين توضع ملفات CLAUDE.md

النطاق	الموقع	الغرض
سياسة إدارة	على لينكس و WSL: /etc/claude-code و CLAUDE.md	تعليمات على مستوى المؤسسة
تعليمات المستخدم	claude/CLAUDE.md/~	تفضيلاتك في كل مشاريعك
تعليمات المشروع	claude/CLAUDE.md/. أو CLAUDE.md/.	تعليمات مشتركة مع الفريق عبر Git
تعليمات محلية	CLAUDE.local.md/.	تفضيلاتك في هذا المشروع، وتضاف إلى .gitignore.

التوثيق يذكر أيضا مسارات السياسة المدارة على أنظمة أخرى: على macOS `/Library/Application Support/` وعلى Windows `C:\Program Files\ClaudeCode\CLAUDE.md` و `ClaudeCode/CLAUDE.md`. وترتيب التحميل من الأوسع نطاقا إلى الأخص، فتظهر تعليمات المشروع في السياق بعد تعليمات المستخدم.

كيف تبدأ

توليد ملف بداية

```
/init
```

التوثيق يقول إن `/init` يحلل مستودعك وينشئ ملفا فيه أوامر البناء وتعليمات الاختبار والأعراف التي اكتشفها. وإذا كان الملف موجودا فإنه يقترح تحسينات بدل أن يستبدله. ثم تحسنه أنت بتعليمات لا يستطيع اكتشافها بنفسه.

كيف تكتب تعليمات فعالة

- **الحجم:** التوثيق يستهدف أقل من 200 سطر لكل ملف، لأن الأطول يستهلك سياقاً أكثر ويقلل الالتزام.
- **البنية:** عناوين ونقاط، لا فقرات كثيفة.
- **التحديد:** «استخدم مسافتين للإزاحة» أفضل من «نسق الكود بشكل صحيح».
- **عدم التناقض:** إذا تناقضت قاعدتان فقد يختار Claude إحداها عشوائياً.

my-project #

أوامر

تشغيل المشروع: npm run dev

تشغيل الاختبارات: npm test

الفحص: npm run lint

أعراف

مسافتان للإزاحة، لا علامات جدولة.

معالجات الـ API في src/api/handlers فقط.

شغل npm test قبل أي commit.

قرارات معمارية

طبقة الوصول للبيانات في src/db وحدها.

ممنوع استدعاء قاعدة البيانات من المكونات مباشرة.

متى تضيف إلى CLAUDE.md

التوثيق يعطي أربع علامات واضحة:

1. عندما يرتكب Claude الخطأ نفسه للمرة الثانية.
2. عندما تكشف مراجعة كود شيئاً كان يجب أن يعرفه عن هذا المستودع.
3. عندما تكتب التصحيح نفسه الذي كتبت في الجلسة الماضية.
4. عندما يحتاج زميل جديد السياق نفسه ليصبح منتجاً.

استيراد ملفات أخرى

يمكن لملف CLAUDE.md استيراد ملفات أخرى بصيغة `@path/to/import`. المسارات النسبية والمطلقة مسموحة، والنسبي يحل بالنسبة للملف الذي يحتويه لا لمجلد العمل. والاستيراد المتداخل مسموح بعمق أقصاه أربع قفزات. ولمنع استيراد مسار ذكرته نصاً، ضعه بين علامتي اقتباس خلفية.

التوثيق واضح: Claude Code يقرأ `CLAUDE.md` لا `AGENTS.md`. إذا كان مستودعك يستخدم `AGENTS.md` لوكلاء آخرين، أنشئ `CLAUDE.md` يستورده بسطر `@AGENTS.md` ثم أضف تحته تعليمات خاصة بـ Claude.

قواعد المسارات في `claude/rules`.

للمشاريع الكبيرة، يمكن تقسيم التعليمات إلى ملفات داخل `.claude/rules/`. وأهم ميزة فيها أن القاعدة يمكن ربطها بمسارات محددة عبر حقل `paths` في مقدمة YAML، فلا تحمل إلا عندما يعمل Claude على ملفات مطابقة.

مقدمة YAML التي تربط القاعدة بمسارات معينة

```
---
paths:
  - "src/api/**/*.ts"
---
```

بقية الملف - `.claude/rules/api.md`

قواعد تطوير الـ API

كل نقطة نهاية تتحقق من مدخلاتها قبل أي معالجة.
استخدم صيغة الخطأ القياسية المعرفة في `src/api/errors`.
أضف تعليقات توثيق OpenAPI لكل نقطة نهاية جديدة.

القواعد التي لا تحمل حقل `paths` تحمل دائماً بالأولوية نفسها لـ `.claude/CLAUDE.md`. والقواعد الشخصية في `~/claude/rules/` تحمل قبل قواعد المشروع، فتكون لقواعد المشروع أولوية أعلى.

الذاكرة التلقائية

الذاكرة التلقائية مفعلة افتراضياً. يحفظ Claude أربعة أنواع من الملاحظات، ويسجل النوع في حقل `type` في مقدمة ملف الذاكرة:

النوع	ماذا يحفظ
user	دورك وخبرتك وتفضيلات عملك
feedback	تصحيحاتك ل Claude J والمقاربات التي أكدتها
project	العمل الجاري والمواعيد والقرارات التي لا تستنتج من الكود أو من سجل Git
reference	أين تجد معلومات خارج المشروع، مثل متتبع مسائل أو لوحة

ويتخطى Claude ما يستطيع استنتاجه من الكود نفسه مثل المعمارية ومسارات الملفات، ويتخطى ما تقوله ملفات CLAUDE.md أصلاً. مكان التخزين `~/ .claude/projects/<project>/memory/` وفيه ملف فهرس `MEMORY.md` وملف لكل موضوع. والذاكرة محلية على الجهاز ولا تشارك عبر الأجهزة.

إدارة الذاكرة

```
/memory
/context
```

إيقاف الذاكرة التلقائية لمشروع واحد

```
{
  "autoMemoryEnabled": false
}
```

ويمكن إيقافها عبر متغير البيئة `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`. وملفات الذاكرة نصوص Markdown عادية تستطيع تحريرها أو حذفها في أي وقت.

إذا لم يلتزم Claude بتعليماتك

1. شغل `/context` وتحقق أن ملفك ظهر تحت Memory files. إذا لم يظهر فهو لا يراه.
2. تأكد أن الملف في موقع يحمل لجلستك.
3. اجعل التعليمات أكثر تحديداً.
4. ابحث عن تعليمات متناقضة بين الملفات.
5. إذا كانت التعليمات يجب أن تنفذ في لحظة محددة — قبل كل commit مثلاً — فاكتبها خطافاً لا تعليمات.

الجلسات: الاستئناف والتفريع وإدارة السياق

الجلسة محادثة محفوظة مرتبطة بمجلد المشروع، تحفظ محليا أثناء عملك، فتستطيع العودة إليها أو تفريعها لتجربة مقارنة أخرى.

الاستئناف

الأمر	ماذا يفعل
claude --continue	استئناف آخر جلسة تفاعلية في المجلد الحالي
claude --resume	فتح منتقي الجلسات
<claude --resume <name	استئناف الجلسة المسماة مباشرة
<claude --from-pr <number	منتقي جلسات مرتبطة بطلب دمج معين
resume/	التبديل إلى محادثة أخرى من داخل جلسة نشطة

ما الذي يستعاد فعلا

- سجل المحادثة كاملا بما فيه استدعاءات الأدوات ونتائجها.
- النموذج الذي كانت الجلسة تستخدمه، إلا في حالات يذكرها التوثيق مثل تقاعد النموذج أو تمرير `--model`.
- وضع الأذونات، بتفصيل يختلف حسب طريقة الاستئناف.
- الهدف النشط والمهام المجدولة التي لم تنته صلاحيتها.

ما لا يستعاد

التوثيق يذكر أن أعلام الإطلاق ليست كلها تستعاد. إذا كانت جلستك تعتمد على `--mcp-config` أو `--` settings أو `--plugin-dir` أو `--fallback-model` أو مجلدات أضيفت بـ `--add-dir`، فمررها مجددا عند الاستئناف. أما ملفات الإعدادات القياسية فتقرأ عند الإطلاق ولا تحتاج تمريرا.

تسمية الجلسات

التسمية عند البدء وأثناء العمل

```
claude -n auth-refactor  
/rename auth-refactor
```

التسمية تجعل الجلسة قابلة للعثور في المنتقي وقابلة للاستئناف بالاسم. وهي الأهم عندما تعمل على عدة مهام بالتوازي.

تفرع محادثة

التفرع ينشئ نسخة من المحادثة حتى الآن وينقلك إليها، ويترك الأصلية كما هي. استخدمه لتجربة مقارنة مختلفة دون خسارة المسار الذي كنت عليه.

التفرع

```
/branch try-streaming-approach  
claude --continue --fork-session
```

إدارة السياق داخل الجلسة

الأمر	ماذا يفعل	متى
context/	عرض ما يستهلك السياق حالياً	عندما تشعر أن الردود تباطأت
compact/ [تعليمات]	استبدال السجل بملخص، مع إمكانية توجيهه	عند اقتراب امتلاء السياق مع رغبتك في المتابعة
clear/ [اسم]	بدء محادثة جديدة بسياق فارغ	عند تغيير الهدف كلياً
export/ [ملف]	تصدير المحادثة نصاً	عند الحاجة لتوثيق ما جرى

ما ينبج من الضغط

التوثيق يذكر أن CLAUDE.md في جذر المشروع ينبج من الضغط: بعد `/compact` يعيد Claude قراءته من القرص وحقنه في الجلسة. أما التعليمات التي أعطيتها في المحادثة فقط فتضيع. إذا كانت تعليمات مهمة، اكتبها في CLAUDE.md.

أين تحفظ الجلسات

افتراضيا يحفظ Claude Code النصوص بصيغة JSONL في `~/.claude/projects/<project>/<session-id>.jsonl`، حيث `<project>` هو مسار مجلد عملك بعد استبدال الرموز غير الأبجدية الرقمية بـ `-`. والتوثيق يحذر من أن صيغة السطور داخلية وتتغير بين الإصدارات، فلا تبين سكريتا يحللها مباشرة، واستخدم `/export` أو الواجهات البرمجية بدلا من ذلك.

مراجعة التغييرات والتراجع عن خطأ

هذا الفصل هو شبكة الأمان. أي شخص يستخدم أداة وكيلا سيمر بلحظة يعدل فيها الوكيل شيئا لم يكن يجب أن يعدل. المهم أن تعرف كيف ترجع.

قاعدة المراجعة قبل الاعتماد

1 راجع الفرق أولا

شغل `/diff` وانظر ما تغير فعلا، لا ما قيل لك إنه تغير.

2 شغل الاختبارات

قبل أي `commit`، لا بعده.

3 اقرأ ما لا تفهمه

إذا وجدت تغييرا لا تفهم سببه، اسأل عنه قبل الاعتماد.

4 اعتمد على دفعات صغيرة

تغيير صغير مراجع أفضل من عشرة ملفات تعتمد عليها دفعة واحدة.

نقاط الاستعادة Checkpointing

التوثيق يفرد صفحة مرجعية ل Checkpointing، ويصفها بأنها إرجاع الكود والمحادثة إلى نقطة سابقة داخل الجلسة نفسها. وهي مختلفة عن التفرع: التفرع ينسخ المحادثة إلى مسار جديد، بينما الاستعادة ترجع بك إلى الوراء. كما تذكر صفحة الجلسات أن قائمة الاستعادة تحتوي مدخلا للجلسة السابقة يتيح الرجوع إلى ما قبل `/clear` داخل نفس عملية Claude Code.

شبكة أمان لا تعتمد على الأداة

أفضل حماية ليست ميزة داخل Claude Code بل عادة في Git. اجعل كل جلسة عمل تبدأ من حالة نظيفة معروفة:

```
git status
git checkout -b feature/my-task
claude
```

```
git diff
git checkout -- src/app.js
git stash
git reset --hard
```

احذر من هذا الأمر تحديداً

`git reset --hard` يحذف كل التغييرات غير المعتمدة نهائياً، وما حذفه لا يسترجع من Git. شغل `git diff` وانظر ماذا ستخسر قبل تنفيذه، أو استخدم `git stash` الذي يحفظ التغييرات بدل أن يمحوها.

حدود مجلد العمل

التوثيق يذكر حماية مهمة: في وضع Manual يستطيع Claude Code الكتابة فقط في المجلد الذي بدأ فيه ومجلداته الفرعية، ولا يستطيع تعديل ملفات في مجلدات أعلى دون إذن صريح. كما يسألك في وضع Manual قبل قراءة مسارات خارج هذا الحد بأدوات القراءة والبحث. ولهذا لا تشغل Claude Code من مجلد المنزل.

العمل مع Git والفروع وطلبات الدمج

Claude Code يعمل مع Git مباشرة: يجهز التغييرات، ويكتب رسائل الـ commit، وينشئ الفروع، ويفتح طلبات الدمج. الفائدة الحقيقية تظهر عندما تجعل ذلك جزءا من سير عمل منضبط لا أمرا عابرا.

سير عمل الفرع الواحد

- 1 افتح فرعاً لكل مهمة
لا تعمل على الفرع الرئيسي مباشرة.
- 2 اطلب خطة قبل التنفيذ
في وضع التخطيط، لتراجع النطاق قبل أن يكتب سطر.
- 3 نفذ ثم راجع
ثم الاختبارات ثم الاعتماد. `/diff`
- 4 commit صغير ومعنوي
رسالة تشرح لماذا لا ماذا.
- 5 افتح طلب دمج
واترك للمراجع فرصة حقيقية للفهم.

طلبات مفيدة مع Git

مراجعة قبل الاعتماد

راجع التغييرات الحالية في شجرة العمل قبل أن أعتمدها.

أعطني:

أولاً. ملخص ما تغير، ملفاً ملفاً، في سطر لكل ملف

ثانياً. أي تغيير لم أطلبه صراحة

ثالثاً. أي تغيير قد يكسر سلوكاً قائماً

رابعاً. ملفات كان يجب تعديلها ولم تعدل

لا تعدل شيئاً الآن. راجع فقط.

اكتب رسالة commit للتغييرات الحالية.

القواعد:

سطر أول لا يتجاوز 72 حرفاً، يقول ماذا تغير

سطر فارغ

فقرة تشرح لماذا كان التغيير ضرورياً، لا كيف نفذ

اذكر أي أثر جانبي أو تغيير في السلوك

لا تذكر أسماء ملفات في السطر الأول.

حل تعارض دمج

ساعدني في حل تعارضات الدمج الحالية.

لكل تعارض:

أولاً. ما الذي كان يحاول كل طرف فعله

ثانياً. هل التعارض نصي فقط أم منطقي

ثالثاً. الحل المقترح وسببه

رابعاً. ما الذي يجب اختباره بعد الحل

إذا كان طرفا التعارض يغيران السلوك نفسه بطريقتين مختلفتين، توقف واسألني.

الأتمتة في التكامل المستمر

التوثيق يذكر أن مراجعة الكود وفرز المسائل يمكن أتمتتهما عبر GitHub Actions أو GitLab CI/CD، وأن هناك مراجعة كود تلقائية على كل طلب دمج عبر GitHub Code Review، وأن ذكر @Claude في Slack يمكن أن يحول بلاغ خطأ إلى طلب دمج.

الاختبارات والفحص والتصحيح وإعادة الهيكلة

هذه هي المهام التي يتفوق فيها الوكيل فعلا، لأن لها معيار نجاح آليا: الاختبار ينجح أو يفشل، والفحص يمر أو لا يمر.

الاختبارات أولا

أفضل طريقة لاستخدام أداة وكيلة في كود قائم هي أن تعطيها معيارا آليا تتحقق به من نفسها. اطلب الاختبار قبل التعديل، لا بعده.

اختبارات قبل التعديل

قبل أن نعدل [الوحدة]، اكتب اختبارات تصف سلوكها الحالي.

المطلوب:

- أولا. اقرأ الوحدة وحدد سلوكها الفعلي، لا المفترض
- ثانيا. اكتب اختبارات تغطي المسار الطبيعي والحدود وحالات الفشل
- ثالثا. شغلها وتأكد أنها تنجح على الكود الحالي كما هو
- رابعا. اذكر أي سلوك بدا لك خاطئا ولم تختبره

لا تعدل كود الوحدة نفسها في هذه الخطوة.

تشخيص خطأ

شخص هذا الخطأ.

رسالة الخطأ كاملة:
[رسالة الخطأ]

السلوك المتوقع:
[المتوقع]
السلوك الفعلي:
[الفعلي]
ما جربته:
[المحاولات]

المطلوب:
أولاً. اقرأ الكود المتعلق وحدد أربع فرضيات مرتبة بالأرجحية
ثانياً. لكل فرضية: كيف نثبتها أو ننفیها بأقل تغيير ممكن
ثالثاً. ابدأ بالاختبار الأسرع ونفذه
رابعاً. أخبرني بالنتيجة قبل أن تنتقل للفرضية التالية
لا تغير كوداً على أمل أن يعمل.

خطأ شائع في التصحيح

لا تسمح بأن تتحول الجلسة إلى سلسلة تغييرات عشوائية حتى يختفي الخطأ. اطلب فرضية ودليلاً قبل كل تغيير.
الخطأ الذي يختفي بلا تفسير يعود.

إعادة الهيكلة

إعادة هيكلة آمنة

أعد هيكلة [الوحدة] بحيث
[الهدف]

الشروط:

لا تغيير في السلوك الخارجي
الاختبارات الحالية يجب أن تبقى ناجحة بلا تعديل
على دفعات: نفذ خطوة واحدة، شغل الاختبارات، ثم توقف وأخبرني
ابدأ بأن تعرض لي خطة الخطوات وحدها، وانتظر موافقتي.

الفحص والتنسيق

اجعل الفحص والتنسيق آليين لا طلبا يدويا. أدرج أوامرهما في CLAUDE.md ليعرفها Claude في كل جلسة، ثم اربطهما بخطاف حتى يعملوا بعد كل تعديل ملف كما ذكر التوثيق في مثال الخطافات.

المراجعة الأمنية

التوثيق يذكر أمر `/security-review` ويصفه بأنه تمريرة أمنية عند الطلب على التغييرات في فرعك الحالي، ويذكر إضافة إرشاد الأمان التي تجعل Claude يراجع ويصلح الثغرات في تغييراته أثناء الجلسة. اجعل واحدة منهما جزءا من روتينك قبل أي طلب دمج.

المهارات Skills والخطافات Hooks

هاتان الآليتان تحولان Claude Code من أداة عامة إلى أداة تعرف طريقة عمل فريقك.

المهارات

التوثيق يصف المهارات بأنها تحزم سير عمل متكررا يمكن لفريقك مشاركته، وتستدعى مثل `/review-pr` أو `/deploy-staging`. والفارق المهم عن القواعد أن المهارة لا تحمل إلا عندما تستدعيها أنت أو عندما يرى Claude أنها متعلقة بطلبك، بينما القواعد تحمل في كل جلسة أو عند فتح ملفات مطابقة.

إذا كانت التعليمات...	ضعها في
حقيقة يجب أن تعرف في كل جلسة	CLAUDE.md
خاصة بنوع ملفات أو مجلد	قاعدة في <code>paths</code> <code>claude/rules</code> بحقل
إجراء متعدد الخطوات يستدعى عند الحاجة	مهارة Skill
يجب أن تنفذ في لحظة محددة مهما قرر Claude	خطاف Hook

الخطافات

الخطافات تشغل أوامر صدفية قبل أو بعد إجراءات Claude Code، مثل التنسيق التلقائي بعد كل تعديل ملف، أو تشغيل الفحص قبل `commit`. وهي الآلية الوحيدة التي تنفذ بغض النظر عما يقرره Claude، ولهذا فالتوثيق يوجهك إليها عندما تريد منعاً حقيقياً لا مجرد توجيه.

إدارة الخطافات

`/hooks`

خطاف أول يستحق الإعداد

خطاف `PreToolUse` يمنع إجراء قبل وقوعه. استخدمه لمنع الكتابة في ملفات حساسة أو تشغيل أوامر محظورة في مشروعك، بدل الاعتماد على تعليمات نصية في `CLAUDE.md` قد لا يلتزم بها.

ويذكر التوثيق خطافا مفيدا للتشخيص اسمه `InstructionsLoaded` ، يسجل أي ملفات تعليمات حملت ومتى ولماذا، وهو مفيد لتصحيح القواعد المرتبطة بمسارات.

الإضافات Plugins

التوثيق يفرد صفحات لاكتشاف الإضافات الجاهزة عبر الأسواق، وإنشاء الإضافات، وإنشاء سوق إضافات وتوزيعه، وتقييد إصدارات اعتماديات الإضافة. تعامل مع أي إضافة أو مهارة من طرف ثالث كما تتعامل مع أي برنامج تنزله: اعرف مصدرها، واقرأ ما تطلبه، ولا تثبت ما لا تحتاجه.

الوكلاء الفرعيون والعمل المتوازي وأشجار العمل

عندما تكبر المهمة، لا تحل بجلسة واحدة أطول بل بتقسيمها. التوثيق يوفر أكثر من آلية لذلك، ولكل واحدة موضعها.

الوكلاء الفرعيون Subagents

التوثيق يذكر إمكانية تشغيل عدة وكلاء يعملون على أجزاء مختلفة من المهمة في وقت واحد، مع وكيل قائد ينسق العمل ويوزع المهام الفرعية ويدمج النتائج. والصفحات المتعلقة تشمل: تشغيل الوكلاء بالتوازي، وإنشاء وكلاء فرعيين مخصصين، وإدارة عدة وكلاء عبر عرض الوكلاء، وتنسيق فرق من جلسات Claude Code، والمراسلة بين الجلسات، وتنسيق الوكلاء الفرعيين على نطاق واسع عبر سير العمل الديناميكي.

ما لا ترثه الوكلاء الفرعية

التوثيق يوضح أن الذاكرة التلقائية للمحادثة الرئيسية لا تحمل في الوكلاء الفرعيين، والاستثناء هو التفرع الذي يرث المحادثة الأصلية ورسالة النظام. وللوكيل الفرعي ذاكرة تلقائية خاصة به في مجلد منفصل إذا فعلت.

أشجار العمل Worktrees

أشجار عمل Git تتيح تشغيل جلسات متوازية معزولة على فروع منفصلة، وللتوثيق صفحة مخصصة لها. الفائدة العملية: تجرب مقاربتين في وقت واحد دون أن تتداخل تغييرات إحداها مع الأخرى، ودون تبديل الفروع في مجلد واحد.

شجرة عمل لكل تجربة

```
git worktree add ../experiment-a -b experiment-a
cd ../experiment-a
claude
```

الذاكرة التلقائية عبر أشجار العمل

التوثيق يذكر أن كل أشجار العمل والمجلدات الفرعية داخل المستودع نفسه تشترك في مجلد ذاكرة تلقائية واحد. أما ملف `CLAUDE.local.md` المستبعد من Git فيوجد فقط في شجرة العمل التي أنشأته فيها.

متى تستخدم كل آلية

الآلية	الحالة
وكلاء فرعيون بالتوازي	مهمة واحدة كبيرة قابلة للتقسيم إلى أجزاء مستقلة
تفريع المحادثة، أو شجرتنا عمل	تجربة مقاربتين مختلفتين للمشكلة نفسها
أشجار عمل منفصلة	العمل على ميزتين مختلفتين في وقت واحد
الأمر / btw	سؤال جانبي لا تريد إضافته للمحادثة

MCP: ربط Claude Code بأدواتك

بروتوكول MCP معيار مفتوح لربط أدوات الذكاء الاصطناعي بمصادر بيانات خارجية. به يستطيع Claude Code قراءة مستندات التصميم في Google Drive، وتحديث تذاكر Jira، وسحب بيانات من Slack، أو استخدام أدواتك الخاصة.

من أين تبدأ

إدارة خوادم MCP

```
/mcp
/mcp reconnect <server>
/mcp disable <server>
```

التوثيق يفرد صفحة بداية سريعة لربط أول خادم من طرف إلى طرف، وصفحة أوسع لربط Claude Code بالأدوات عبر MCP، وصفحة إدارية للتحكم في وصول خوادم MCP على مستوى المؤسسة.

أمان MCP

ما يقوله التوثيق بوضوح

قائمة خوادم MCP المسموحة تضبط داخل كود مصدر مشروعك كجزء من إعدادات Claude Code التي يودعها المهندسون في نظام التحكم بالإصدارات.

التوثيق يشجع على كتابة خوادمك بنفسك أو استخدام خوادم من مزودين تثق بهم، ويذكر صراحة أن Anthropic تراجع الموصلات مقابل معايير الإدراج قبل إضافتها إلى الدليل، لكنها لا تدقق أمنيًا ولا تدير أي خادم MCP.

الخوادم الجديدة تتطلب تحقق ثقة عند أول استخدام. ولاحظ أن التحقق معطل عند التشغيل غير التفاعلي بعلم

- p .

قاعدة عملية

- لا تضيف خادما لا تعرف ماذا يقرأ وماذا يكتب.

- أعط كل خادم أقل صلاحية تكفي مهمته.
- راجع قائمة الخوادم المفعلة دوريا وأوقف ما لم تعد تستخدمه.
- على مستوى الفريق، اجعل قائمة الخوادم مراجعة في طلب دمج مثل أي كود.

الأمان: الأسرار ومراجعة الأوامر والمحتوى غير الموثوق

أداة تشغل أوامر في جهازك تحتاج انضباطا أمنيا. هذا الفصل يجمع ما يقوله التوثيق الرسمي، لا نصائح عامة.

الحمايات المدمجة

- نموذج قائم على الأذونات: في وضع Manual يبدأ Claude Code بصلاحيات قراءة فقط، ويسأل قبل تعديل ملف أو تشغيل أمر.
- أوامر قراءة فقط: يشغل مجموعة مدمجة مثل `ls` و `cat` و `git status` دون سؤال.
- حد مجلد العمل: لا يكتب خارج المجلد الذي بدأ فيه ومجلداته الفرعية دون إذن.
- صندوق Bash الرملي: عزل نظام الملفات والشبكة لأوامر Bash، يضبط بـ `/sandbox`.
- تحقق الثقة: أول تشغيل على مستودع وأي خادم MCP جديد يتطلبان تحقق ثقة.
- تخزين آمن للاعتمادات: المفاتيح والرموز تخزن في سلسلة مفاتيح macOS عند توفرها، وتحمي بصلاحيات الملفات على Windows و Linux.

حقن التعليمات Prompt Injection

حقن التعليمات أسلوب يحاول به مهاجم تجاوز تعليمات المساعد أو التلاعب بها عبر إدراج نص خبيث. التوثيق يذكر عدة حمايات، من أهمها عمليا:

- الأوامر التي تجلب محتوى من الويب مثل `curl` و `wget` لا يوافق عليها تلقائيا افتراضيا. في وضع Manual تسأل مثل أي أمر غير قرأني. ولمنعها كليا أضفها إلى `permissions.deny`.
- جلب الويب يستخدم نافذة سياق منفصلة لتجنب حقن تعليمات خبيثة في المحادثة.
- في وضع Manual، الأوامر المشبوهة تتطلب موافقة يدوية حتى لو كانت في قائمة السماح سابقا.

أفضل الممارسات مع المحتوى غير الموثوق

راجع الأوامر المقترحة قبل الموافقة.

تجنب تمرير محتوى غير موثوق مباشرة إلى Claude.

تحقق من التغييرات المقترحة على الملفات الحرجة.

استخدم أجهزة افتراضية لتشغيل السكريبتات واستدعاء الأدوات، خصوصا عند التعامل مع خدمات ويب خارجية.

بلغ عن السلوك المريب بـ `/feedback`.

والتوثيق نفسه يذيل ذلك بتحذير مهم: رغم أن هذه الحماية تقلل المخاطر بدرجة كبيرة، لا يوجد نظام محصن تماما ضد كل الهجمات.

الأسرار: القاعدة التي لا تكسر

1. لا تكتب مفتاحا ولا كلمة مرور ولا رمزا داخل الكود أبدا.
2. ضعها في ملف `.env` واقرأها من متغيرات البيئة.
3. أضف `.env` إلى `.gitignore` قبل أن تنشئه، لا بعده.
4. أنشئ `.env.example` فيه أسماء المتغيرات بلا قيم، وأودعه في المستودع.
5. إذا تسرب مفتاح إلى Git فلا يكفي حذفه في commit جديد: أبطل المفتاح وأنشئ غيره.

gitignore — الحد الأدنى

```
.env
.env.local
*.pem
*.key
secrets/
CLAUDE.local.md
```

env.example — يودع في المستودع

```
DATABASE_URL=
API_KEY=
JWT_SECRET=
```

قبل أول commit في أي مشروع

شغل `git status` وتأكد أن `.env` غير مدرج. تسريب مفتاح إلى مستودع عام يكتشف آليا خلال دقائق من قبل من يبحث عنه.

العمل على كود حساس

- راجع كل تغيير مقترح قبل الموافقة.
- استخدم إعدادات أذونات خاصة بالمستودعات الحساسة.
- فكر في حاويات التطوير dev containers لعزل إضافي.
- دقق إعدادات أذوناتك دوريا بـ `/permissions`.

إدارة السياق والمشاريع الكبيرة

المشروع الكبير لا يدار بجلسة أطول. يدار بتقليل ما يدخل السياق وبتقسيم العمل.

ماذا يملأ سياقك

شغل `/context` لترى ما يستهلك السياق حالياً. التوثيق يوضح أن ملفات CLAUDE.md تحمل في بداية كل جلسة وتستهلك رموزاً إلى جانب محادثتك، ولهذا يستهدف أقل من 200 سطر لكل ملف. والقواعد المرتبطة بمسارات تحمل فقط عندما يعمل Claude على ملفات مطابقة، وهي الوسيلة الرئيسية لتقليل الضجيج في مستودع كبير.

خمس عادات تقلل استهلاك السياق

1. جلسة واحدة لهدف واحد. عند تغير الهدف، `/clear`.
2. اطلب من Claude قراءة ملفات محددة بدل استكشاف المستودع كله.
3. استخدم قواعد المسارات بدل ملف CLAUDE.md ضخمة.
4. حول الإجراءات المتكررة إلى مهارات تحمل عند الحاجة فقط.
5. استخدم `/compact` مع تعليمات موجهة بدل تركه يلخص كل شيء بالتساوي.

ضغط موجه بدل ضغط عام

```
/compact keep architecture decisions and edited file names; drop debugging detail
```

المستودعات الكبيرة

التوثيق يفرد صفحة لإعدادات Claude Code في مستودع أحادي monorepo أو قاعدة كود كبيرة، ويذكر إعدادات `claudeMdExcludes` لتخطي ملفات CLAUDE.md الخاصة بفرق أخرى لا تخص عملك. والأنماط تطابق على المسارات المطلقة بصيغة glob، ويمكن ضبطها في أي طبقة إعدادات.

```
{
  "claudeMdExcludes": [
    "**/monorepo/CLAUDE.md",
    "/home/user/monorepo/other-team/.claude/rules/**"
  ]
}
```

ما لا يمكن استبعاده

التوثيق يذكر أن ملفات CLAUDE.md الخاصة بسياسة مدارة لا يمكن استبعادها، لضمان سريان تعليمات المؤسسة بغض النظر عن الإعدادات الفردية.

تقسيم المهمة الكبيرة

1 اكتب النطاق أولاً

ما الداخل وما الخارج، في ملف تعود إليه.

2 قسم إلى مراحل لكل مرحلة مخرج

لا إلى مهام بلا مخرج ملموس.

3 جلسة لكل مرحلة

وسمها باسم المرحلة لتجدها لاحقاً.

4 سجل القرارات في CLAUDE.md

حتى لا تنقض في المرحلة التالية.

5 راجع بعد كل مرحلة

قبل أن تفتح المرحلة التي بعدها.

سير عمل كامل: من البيئة إلى الإصدار

هذه المراحل العشر هي الهيكل الذي تعمل به على أي مشروع حقيقي. لكل مرحلة مخرج ملموس ومعياري انتقال.

المرحلة صفر: البيئة

- ثبت Claude Code وتحقق بـ `claude doctor`.
- تأكد أن المشروع في مستودع Git وأن الشجرة نظيفة.
- أنشئ `.gitignore` و `.env.example` قبل أي كود.
- معيار الانتقال: تستطيع تشغيل المشروع واختباره محليا.

المرحلة الأولى: المتطلبات

استخراج المتطلبات

سنبني:

[وصف المشروع]

المستخدم:

[المستخدم]

قبل أي كود، ساعدني في تحديد:

أولا. المشكلة في ثلاث جمل

ثانيا. قصص المستخدم الأساسية، خمس على الأكثر

ثالثا. ما هو خارج النطاق صراحة

رابعا. معايير القبول: متى نعتبر المشروع منتهيا

خامسا. الأسئلة التي يجب أن أجيب عنها قبل البدء

اكتب النتيجة في ملف `REQUIREMENTS.md`.

لا تكتب أي كود في هذه المرحلة.

المرحلة الثانية: المعمارية

قرار معماري

اقترح معمارية لهذا المشروع بناء على REQUIREMENTS.md.

المطلوب:

- أولا. خياران معماريان مختلفان جوهريا
- ثانيا. لكل خيار: ما يناسبه، وأين يفشل، وتكلفة التراجع عنه
- ثالثا. توصية واحدة، والشرط الذي يقبلها
- رابعا. بنية المجلدات المقترحة
- خامسا. القرارات التي يجب تثبيتها في CLAUDE.md

قيودي:

[القيود]

خبرتي:

[الخبرة]

المرحلة الثالثة: الأساس

- أنشئ الهيكل والإعدادات وملف CLAUDE.md.
- شغل `/init` ثم حسن الملف يدويا.
- أضف الفحص والتنسيق والاختبارات من اليوم الأول.
- معيار الانتقال: هيكل يعمل، واختبار واحد ناجح ولو تافها.

المرحلة الرابعة: الميزات الأساسية

- ميزة واحدة لكل فرع ولكل جلسة.
- خطة قبل التنفيذ، ثم `/diff` بعده، ثم الاختبارات، ثم `commit`.
- معيار الانتقال: كل قصة مستخدم أساسية تعمل من طرف إلى طرف.

المرحلة الخامسة: الاختبارات

- غط الحدود وحالات الفشل لا المسار السعيد فقط.
- اجعل الاختبارات تعمل في التكامل المستمر لا على جهازك فقط.

- معيار الانتقال: تستطيع تعديل الكود وأنت واثق أن الاختبار سيمسك الانحدار.

المرحلة السادسة: تجربة الاستخدام

- رسائل الخطأ الظاهرة للمستخدم: مفهومة وتخبره ماذا يفعل.
- الحالات الفارغة وحالات التحميل والفشل.
- معيار الانتقال: شخص لم ير المشروع يستطيع إتمام المهمة الأساسية.

المرحلة السابعة: الأمان

- لا أسرار في الكود، و `.env` مستبعد من Git.
- تحقق من كل مدخل خارجي.
- شغل `/security-review` على تغييرات الفرع.
- معيار الانتقال: لا ثغرة حرجة مفتوحة.

المرحلة الثامنة: البناء

- بناء نظيف من مستودع نظيف، لا من جهازك فقط.
- وثق أوامر البناء في `CLAUDE.md` وفي `README`.
- معيار الانتقال: يستطيع شخص آخر بناء المشروع باتباع التوثيق.

المرحلة التاسعة: مراجعة الإصدار

- اختبر على بيئة قريبة من الإنتاج.
- راجع سجل التغييرات وتأكد أنه يعكس ما تغير فعلا.
- جهز خطة تراجع: كيف ترجع إذا فشل الإصدار.
- معيار الانتقال: تعرف بالضبط ماذا ستفعل لو انكسر شيء بعد ساعة من النشر.

قاعدة المراحل

لا تنتقل لمرحلة قبل تحقق معيار انتقال السابقة. تخطي مرحلة لا يوفر وقتا، بل يؤجل اكتشاف المشكلة إلى مرحلة أعلى.

ورقة المراجعة السريعة

صفحتان ترجع إليهما بعد أن تنسى التفاصيل.

التثبيت والتحديث

```
curl -fsSL https://claude.ai/install.sh | bash
irm https://claude.ai/install.ps1 | iex

brew install --cask claude-code
winget install Anthropic.ClaudeCode

brew upgrade claude-code
winget upgrade Anthropic.ClaudeCode
npm install -g @anthropic-ai/claude-code@latest

claude doctor
```

الجلسات

```
claude
claude -n <session-name>
claude --continue
claude --resume
claude --resume <session-name>
claude --continue --fork-session
claude --from-pr <number>
```


الوظيفة	الأمر
إنشاء CLAUDE.md للمشروع	init/
تحرير ملفات الذاكرة وتبديل الذاكرة التلقائية	memory/
عرض ما يستهلك السياق	context/
تلخيص المحادثة لتفريغ السياق	compact/ [تعليمات]
بدء محادثة جديدة بسياق فارغ	clear/ [اسم]
تفريع المحادثة	branch/ [اسم]
التبديل إلى محادثة أخرى	resume/ [اسم]
تسمية الجلسة الحالية	rename/ [اسم]
مراجعة تغييرات شجرة العمل	diff/
إدارة قواعد السماح والسؤال والمنع	permissions/
عرض إعدادات الخطافات	hooks/
إدارة خوادم MCP	mcp/
إدارة إعدادات الوكلاء الفرعيين	agents/
تبديل النموذج	model/ [النموذج]
ضبط مستوى الجهد	effort/ [المستوى]
الدخول في وضع التخطيط	plan/ [وصف]
فحص إعدادات يشخص ويصلح	doctor/
تصدير المحادثة نصا	export/ [ملف]
تمريرة أمنية على تغييرات الفرع	security-review/
عرض الاستهلاك	usage/

```
claude --permission-mode default
claude --permission-mode plan
claude --permission-mode auto
claude --permission-mode dontAsk
```

روتين الجلسة الآمنة

1. `git status` — ابدأ من شجرة نظيفة.
2. `git checkout -b` — فرع لكل مهمة.
3. وضع التخطيط أولاً، وراجع الخطأ.
4. نفذ، ثم `/diff`.
5. شغل الاختبارات والفحص.
6. `commit` صغير برسالة تشرح السبب.

أخطاء تجنبها

- تشغيل Claude Code من مجلد المنزل بدل مجلد المشروع.
- استخدام `--dangerously-skip-permissions` خارج بيئة معزولة.
- اعتماد تغييرات بلا `/diff` وبلا اختبارات.
- ملف `CLAUDE.md` ضخم بدل قواعد مرتبطة بمسارات.
- جلسة واحدة طويلة لعدة أهداف مختلفة.
- إضافة خادم MCP أو مهارة من مصدر لا تعرفه.
- `sudo npm install -g` أو `npm update -g` لتحديث الأداة.

الخلاصة والخطوة التالية

أهم ما في هذا الدليل ليس قائمة الأوامر بل ثلاث عادات: أن تبدأ كل مهمة من فرع نظيف، وأن تراجع الفروق قبل الاعتماد، وأن تكتب في CLAUDE.md كل ما اضطررت لشرحه مرتين.

خطة الأسبوع الأول

1 اليوم الأول

ثبت الأداة، وشغل `claude doctor`، وافتح مشروعاً حقيقياً واطلب شرح بنيته.

2 اليوم الثاني

شغل `/init` وحسن ملف CLAUDE.md يدوياً.

3 اليوم الثالث

نفذ تعديلاً صغيراً في وضع التخطيط، وراجع بـ `/diff` قبل الاعتماد.

4 اليوم الرابع

اضبط `.gitignore` و `.env.example`، وراجع `/permissions`.

5 اليوم الخامس

اكتب أول قاعدة في `.claude/rules` مرتبطة بمسار.

6 اليومان السادس والسابع

نفذ ميزة كاملة بسير عمل المراحل، من الخطوة إلى طلب الدمج.

بقية السلسلة

- تعلم البرمجة مع Claude Code — لمن يريد أن يتعلم فعلاً لا أن يكتب له.
- ابن موقعك الأول مع Claude Code — مشروع ويب كامل خطوة بخطوة.
- ابن تطبيقك الأول مع Claude Code — تطبيق هاتف من الفكرة إلى البناء.

تنبيه عن التاريخ

Claude Code يتطور أسرع من معظم الأدوات. كل أمر وكل اسم ميزة في هذا الدليل مأخوذ من التوثيق الرسمي على code.claude.com بتاريخ 6 سبتمبر 2026. قبل الاعتماد على سلوك دقيق، راجع صفحة الميزة الرسمية.

برومبتات عربية

هذا المنتج من إنتاج قناة برومبتات عربية.
لمتابعة الأدلة والبرومبتات الجديدة:

<https://t.me/PromptsArabic>